

edit.js

Tuyệt vời! Để thêm chức năng **chỉnh sửa (Edit/Update)** sản phẩm, chúng ta cần cập nhật 3 file chính: **Controller**, **Router** và tạo thêm file **View** để hiển thị form chỉnh sửa.

Dưới đây là các bước chi tiết để thêm chức năng này:

1. Cập nhật `controllers\productController.js`

Chúng ta cần thêm hai hàm mới vào Controller:

1. `getEditFormProduct` : Để hiển thị form chỉnh sửa (dùng phương thức GET).
2. `updateProduct` : Để xử lý dữ liệu từ form và cập nhật vào CSDL (dùng phương thức POST).

✓ Code mới cho `controllers\productController.js`

```
const Product = require("../models/Product")
const fs = require("fs") // Dùng để xóa file ảnh cũ khi cập nhật

// 1. Hàm hiển thị form chỉnh sửa
const getEditFormProduct = async (req, res) => {
  const { _id } = req.params;
  // Tìm sản phẩm theo ID
  const product = await Product.findById(_id);
  if (!product) {
    return res.redirect("/products?msg=not-found"); // Xử lý lỗi nếu không tìm thấy
  }
  // Render form edit, truyền dữ liệu sản phẩm hiện tại
  res.render("edit", { product });
}

// 2. Hàm xử lý cập nhật sản phẩm
const updateProduct = async (req, res) => {
  const { _id } = req.params;
  const data = req.body;

  // Lấy thông tin sản phẩm cũ để kiểm tra ảnh
  const oldProduct = await Product.findById(_id);

  // Kiểm tra nếu có file ảnh mới được upload
  if (req.file) {
    data.image = req.file.filename; // Lưu tên ảnh mới

    // Xóa file ảnh cũ (nếu tồn tại)
```

```

    if (oldProduct.image) {
      const oldImagePath = `./public/uploads/${oldProduct.image}`;
      // Sử dụng fs.unlink để xóa file
      fs.unlink(oldImagePath, (err) => {
        if (err) console.error("Error deleting old image:", err);
      });
    }
  } else {
    // Nếu không có ảnh mới, giữ lại tên ảnh cũ
    data.image = oldProduct.image;
  }

  // Cập nhật sản phẩm trong CSDL
  await Product.findByIdAndUpdate(_id, data, { new: true })
    .then(result => {
      res.redirect("/products?msg=update-success");
    })
    .catch(err => {
      res.redirect("/products?msg=update-failed");
    });
}

// Giữ nguyên các hàm cũ...
const getAllProduct = async (req, res) => {
  // ... (Hàm này giữ nguyên)
}

const getFormProduct = async (req, res) => {
  // ... (Hàm này giữ nguyên)
}

const createProduct = async (req, res) => {
  // ... (Hàm này giữ nguyên)
}

const deleteProduct = async (req, res) => {
  // ... (Hàm này giữ nguyên)
}

module.exports = {
  getAllProduct,
  getFormProduct,
  createProduct,
  deleteProduct,
  // THÊM CÁC HÀM MỚI VÀO EXPORT
  getEditFormProduct,
  updateProduct
}

```

2. Cập nhật `routes\productRouter.js`

Chúng ta cần thêm các route tương ứng với hai hàm Controller mới.

 **Code mới cho** `routes\productRouter.js`

```

const express = require("express")
const upload = require("../middlewares/uploadFiles");

const {
  getAllProduct,
  getFormProduct,
  createProduct,
  deleteProduct,
  // THÊM CÁC HÀM MỚI
  getEditFormProduct,
  updateProduct
} = require("../controllers/productController");

const productRouter = express.Router()

// ... (Các route cũ giữ nguyên) ...
productRouter.get("/", getAllProduct)
productRouter.get("/add", getFormProduct)
productRouter.post("/add", upload.single("image"), createProduct);
productRouter.get("/delete/:_id", deleteProduct)

// 1. Hiển thị form chỉnh sửa (GET)
// Đường dẫn động: /products/edit/ID_SAN_PHAM
productRouter.get("/edit/:_id", getEditFormProduct);

// 2. Xử lý dữ liệu cập nhật (POST)
// Sử dụng cùng đường dẫn nhưng phương thức POST, và áp dụng middleware upload
productRouter.post("/edit/:_id", upload.single("image"), updateProduct);

module.exports = productRouter;

```

3. Tạo file View `views/edit.ejs`

File này gần giống với `add.ejs`, nhưng có hai điểm khác biệt quan trọng:

1. **Thuộc tính `action`**: Chỉ đến route POST mới (`/products/edit/ID`).
2. **Giá trị mặc định (`value`)**: Các trường input phải hiển thị dữ liệu sản phẩm hiện tại.

Code cho `views/edit.ejs`

```

<!DOCTYPE html >
<html lang="en">

<head>
  <title>Edit Product Form</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

```

```

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</head>

<body>

  <div class="container mt-3">
    <h2>Edit Product: <%= product.name %></h2>
    <form action="/products/edit/<%= product._id %>" method="POST" enctype="multipart/form-data">

      <div class="mb-3 mt-3">
        <label for="name">Name:</label>
        <input type="text" class="form-control" id="name" placeholder="Enter name" name="name" value="
<%= product.name %>">
      </div>

      <div class="mb-3 mt-3">
        <label for="price">Price:</label>
        <input type="number" class="form-control" id="price" placeholder="Enter price" name="price" value="
<%= product.price %>">
      </div>

      <div class="mb-3 mt-3">
        <label for="current_image">Current Image:</label>
        <% if (product.image) { %>
          
        <% } else { %>
          <p>No current image.</p>
        <% } %>
      </div>

      <div class="mb-3 mt-3">
        <label for="image">Upload New Image (Optional):</label>
        <input type="file" class="form-control" id="image" name="image" accept="image/*">
      </div>

      <button type="submit" class="btn btn-warning">Update Product</button>
      <a href="/products" class="btn btn-secondary">Cancel</a>

    </form>
  </div>

</body>

</html>

```